Pruebas de integración

Nombre del proyecto	ADOPTAFACIL
Nombre del sistema	Sistema de información para la adopción de animales
Versión	1.0
Equipo	Nicolas Alberto Valenzuela Pacheco Brayan Esteban Sanchez Avila Andres Felipe Gamez Arias Midred Callejas Chaparro
Instructor	Ruben Sanabria

1. Introducción
PARTE I. Fundamentos teóricos de las pruebas de integración
2. Definición de pruebas de integración
3. Objetivos de las pruebas de integración
    3.1. Objetivo general
    3.2. Objetivos específicos
4. Arquitectura del sistema bajo prueba
    4.1. Componentes principales
5. Herramientas utilizadas
    5.1. Laravel Testing Framework
    5.2. PestPHP
PARTE II. Implementación de pruebas de integración en AdoptaFácil
6. Estructura de pruebas del proyecto
7. Pruebas de autenticación de usuarios
    7.1. Descripción
    7.2. Casos de prueba
    7.3. Ejemplo de código
8. Pruebas de registro de usuarios
    8.1. Descripción
    8.2. Casos de prueba

8.3. Ejemplo de código
9. Pruebas de recuperación de contraseña
9.1. Casos de prueba
10. Pruebas de acceso al dashboard
10.1. Ejemplo de código
11. Pruebas del middleware de autenticación
11.1. Caso de prueba
11.2. Ejemplo de código
12. Pruebas de estadísticas del sistema
12.1. Ejemplo de código
13. Validaciones realizadas
14. Resultados obtenidos
15. Beneficios de las pruebas de integración
16. Conclusiones

---

1. Introducción

En el desarrollo de software moderno, la verificación del correcto funcionamiento de los distintos módulos que conforman una aplicación es una tarea fundamental para garantizar la calidad del sistema. Las **pruebas de integración** permiten evaluar la interacción entre múltiples componentes, verificando que estos trabajen correctamente cuando se combinan.

El presente documento describe el proceso de **implementación y ejecución de pruebas de integración en el sistema AdoptaFácil**, una aplicación web desarrollada con el framework **Laravel**. Estas pruebas han sido implementadas utilizando el sistema de testing nativo de Laravel junto con **PestPHP**, lo que permite validar el comportamiento del sistema mediante simulación de solicitudes HTTP y verificación de respuestas.

Las pruebas de integración del sistema se enfocan en validar funcionalidades críticas como:

- Autenticación de usuarios.
- Registro de nuevos usuarios.
- Recuperación de contraseña.
- Acceso al panel principal (dashboard).

- Generación de estadísticas del sistema.
- Validación del middleware de autenticación.
- Interacción entre rutas, controladores y base de datos.

Este documento tiene como propósito presentar el diseño, ejecución y resultados de dichas pruebas, proporcionando una visión completa del proceso de validación del sistema.

PARTE I. Fundamentos teóricos de las pruebas de integración

2. Definición de pruebas de integración

Las **pruebas de integración** son un tipo de prueba de software que se utilizan para verificar la correcta interacción entre diferentes componentes o módulos de una aplicación.

Mientras que las **pruebas unitarias** se enfocan en validar componentes individuales, las pruebas de integración buscan comprobar que dichos componentes funcionen correctamente cuando se integran entre sí.

En aplicaciones web modernas, estas pruebas suelen validar interacciones entre:

- Rutas del sistema.
- Controladores.
- Modelos.
- Middleware.
- Base de datos.
- Servicios externos.

El objetivo principal es garantizar que el flujo completo de una funcionalidad opere de manera correcta.

3. Objetivos de las pruebas de integración

3.1. Objetivo general

Verificar el correcto funcionamiento de las interacciones entre los diferentes módulos del sistema AdoptaFácil mediante pruebas automatizadas de integración.

3.2. Objetivos específicos

- Validar el flujo completo de autenticación de usuarios.
- Verificar el proceso de registro de nuevos usuarios.
- Comprobar el funcionamiento del sistema de recuperación de contraseña.
- Validar el acceso al dashboard del sistema.
- Confirmar la generación correcta de estadísticas.
- Verificar el funcionamiento del middleware de autenticación.
- Validar la interacción entre rutas, controladores y base de datos.

4. Arquitectura del sistema bajo prueba

El sistema AdoptaFácil sigue la arquitectura **MVC (Model-View-Controller)** proporcionada por el framework Laravel.

4.1. Componentes principales

- **Rutas (Routes):** definen los endpoints del sistema y dirigen las solicitudes HTTP hacia los controladores correspondientes.
- **Controladores (Controllers):** contienen la lógica del sistema encargada de procesar las solicitudes y generar respuestas.
- **Modelos (Models):** representan las entidades del sistema y gestionan la interacción con la base de datos.
- **Middleware:** gestiona la seguridad y validación de solicitudes antes de que lleguen a los controladores.
- **Base de datos:** almacena la información persistente del sistema.

Las pruebas de integración se enfocan en validar el funcionamiento conjunto de estos componentes.

5. Herramientas utilizadas

Las pruebas de integración del proyecto fueron implementadas utilizando las siguientes herramientas.

5.1. Laravel Testing Framework

Laravel incluye un sistema de pruebas integrado que permite simular solicitudes HTTP y validar respuestas del sistema.

Este framework permite:

- Simular peticiones GET y POST.
- Validar respuestas HTTP.
- Interactuar con la base de datos de pruebas.
- Probar middleware y autenticación.

5.2. PestPHP

PestPHP es un framework de testing que ofrece una sintaxis más clara y legible para escribir pruebas en PHP.

Ventajas:

- Sintaxis simplificada.
- Integración con PHPUnit.
- Compatibilidad completa con Laravel.

Ejemplo de prueba básica:

```
test('home page loads correctly', function () {
    $response = $this->get('/');
    $response->assertStatus(200);
});
```

PARTE II. Implementación de pruebas de integración en AdoptaFácil

6. Estructura de pruebas del proyecto

Las pruebas de integración se encuentran organizadas dentro del directorio `tests/Feature`.

Estructura general:

```
tests
├── Feature
│   ├── Auth
│   │   ├── AuthenticationTest.php
│   │   ├── RegistrationTest.php
│   │   └── PasswordResetTest.php
│   │
│   ├── DashboardTest.php
│   ├── EstadisticasTest.php
│   └── SettingsTest.php
```

Cada archivo contiene pruebas relacionadas con una funcionalidad específica del sistema.

7. Pruebas de autenticación de usuarios

7.1. Descripción

Estas pruebas verifican que los usuarios puedan iniciar sesión correctamente en el sistema utilizando credenciales válidas. También se valida que el sistema rechace credenciales incorrectas.

7.2. Casos de prueba

ID	Caso	Resultado esperado
INT-01	Login con credenciales válidas	Usuario autenticado
INT-02	Login con credenciales inválidas	Acceso denegado

7.3. Ejemplo de código

```
test('users can authenticate using valid credentials', functi
on () {
    $user = User::factory()->create([
        'password' => bcrypt('password'),
    ]);

    $response = $this->post('/login', [
        'email' => $user->email,
        'password' => 'password',
    ]);

    $response->assertRedirect('/dashboard');
});
```

8. Pruebas de registro de usuarios

8.1. Descripción

Estas pruebas verifican que nuevos usuarios puedan registrarse correctamente en el sistema.

8.2. Casos de prueba

ID	Caso	Resultado esperado
INT-03	Registro con datos válidos	Usuario creado
INT-04	Registro con datos inválidos	Error de validación

8.3. Ejemplo de código

```
test('new users can register', function () {
    $response = $this->post('/register', [
        'name' => 'Usuario Test',
        'email' => 'test@example.com',
        'password' => 'password',
```

```
        'password_confirmation' => 'password',
    ]);

    $response->assertRedirect('/dashboard');
});
```

9. Pruebas de recuperación de contraseña

Estas pruebas validan el sistema de recuperación de contraseña.

9.1. Casos de prueba

ID	Caso	Resultado esperado
INT-05	Solicitud de recuperación	Email enviado
INT-06	Restablecimiento de contraseña	Contraseña actualizada

10. Pruebas de acceso al dashboard

El dashboard es una de las secciones principales del sistema.

Las pruebas verifican que:

- Usuarios autenticados puedan acceder.
- Usuarios no autenticados sean redirigidos.

10.1. Ejemplo de código

```
test('authenticated users can access dashboard', function ()
{
    $user = User::factory()->create();

    $response = $this->actingAs($user)->get('/dashboard');
    $response->assertStatus(200);
});
```

11. Pruebas del middleware de autenticación

El middleware `auth` se encarga de restringir el acceso a ciertas rutas del sistema.

11.1. Caso de prueba

ID	Caso	Resultado esperado
INT-07	Usuario no autenticado accede a dashboard	Redirección a login

11.2. Ejemplo de código

```
test('guest users cannot access dashboard', function () {
    $response = $this->get('/dashboard');
    $response->assertRedirect('/login');
});
```

12. Pruebas de estadísticas del sistema

Estas pruebas validan que el sistema genere correctamente estadísticas relacionadas con:

- Número de mascotas registradas.
- Número de adopciones.
- Información del sistema.

12.1. Ejemplo de código

```
test('statistics endpoint returns correct data', function ()
{
    $user = User::factory()->create();

    $response = $this->actingAs($user)->get('/estadisticas');
    $response->assertStatus(200);
});
```

13. Validaciones realizadas

Durante las pruebas se verificaron múltiples aspectos del sistema.

Validación	Resultado
Flujo completo de autenticación	Correcto
Registro de usuarios	Correcto
Middleware de autenticación	Correcto
Acceso al dashboard	Correcto
Generación de estadísticas	Correcto
Interacción con base de datos	Correcta

14. Resultados obtenidos

Las pruebas de integración ejecutadas sobre el sistema arrojaron los siguientes resultados.

Tipo de prueba	Resultado
Autenticación	Correcto
Registro	Correcto
Recuperación de contraseña	Correcto
Dashboard	Correcto
Middleware	Correcto
Estadísticas	Correcto

No se detectaron errores críticos durante la ejecución de las pruebas.

15. Beneficios de las pruebas de integración

La implementación de pruebas de integración proporciona múltiples beneficios:

- Validación de flujos completos del sistema.
- Detección temprana de errores entre módulos.
- Mayor confiabilidad del software.
- Mejora en la calidad del código.
- Mayor seguridad en el despliegue del sistema.

16. Conclusiones

Las pruebas de integración implementadas en el sistema AdoptaFácil permitieron verificar el correcto funcionamiento de los principales flujos del sistema, incluyendo autenticación, registro de usuarios y acceso a funcionalidades protegidas.

El uso de herramientas como **Laravel Testing y PestPHP** facilitó la creación de pruebas automatizadas que permiten validar la interacción entre rutas, controladores y base de datos.

Los resultados obtenidos demuestran que:

- El sistema maneja correctamente los procesos de autenticación y registro.
- El middleware de seguridad funciona adecuadamente.
- Los endpoints del sistema responden de manera correcta.

La implementación de estas pruebas constituye un componente esencial dentro del proceso de aseguramiento de la calidad del software y garantiza un mayor nivel de confiabilidad en el sistema AdoptaFácil.